

OctWave 3.0: Kaggle Challenge 02

Tom & Jerry Image Classification

Author: Yasandu Kethmika

Team Name: Oct_Endurance

Team Members: Nimsara Kodithuwakku (Team Leader), Yasandu Kethmika,
Pasan Kaluarachchi, Akash Jayasinghe

1. Dataset & Approach

- **Dataset Overview:** The dataset comprises 5,481 images (2,680 training, 2,798 test) combined in a single directory. Labels were mapped via a CSV-driven `tf.data` pipeline rather than a standard directory structure. No corrupt or unreadable files were detected.
- **Class Imbalance:** Exploratory Data Analysis revealed a highly imbalanced distribution. The `tom_only` class acts as the majority (1,252 images), while the `both` class is severely underrepresented (219 images), yielding a $5.72\times$ imbalance ratio.
- **Evaluation Metric:** The Macro F1-score was strictly utilized to evaluate performance, ensuring that minority classes influence the final score equally alongside majority classes.

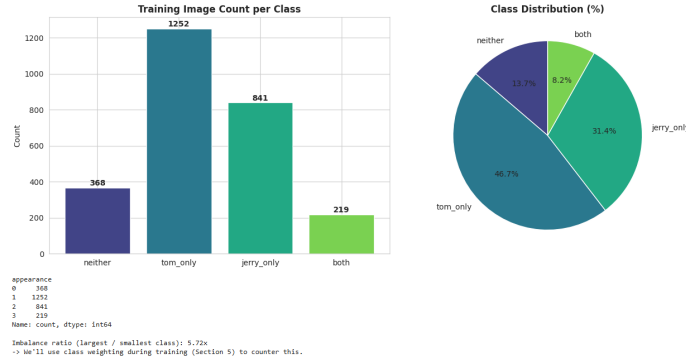


Figure 1: Class Distribution and Imbalance Ratio

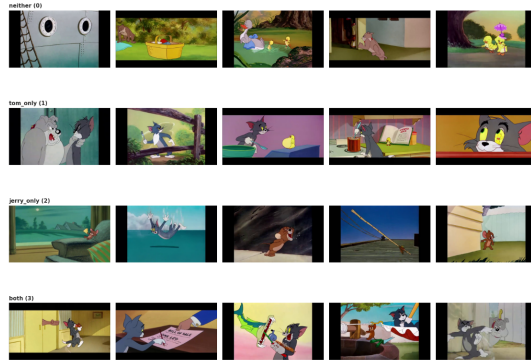


Figure 2: Sample Images per Class

2. Preprocessing & Data Pipeline

- **Validation Strategy:** To preserve the exact class distribution for reliable Macro F1 calculation, an 85/15 Stratified train-validation split was implemented. This prevented the starvation of the `both` class in the validation set.
- **Augmentation:** To improve generalization against noise, motion blur, and partial occlusions, a sequential augmentation layer was applied exclusively to the training set: Random Horizontal Flip, Random Rotation (0.08), Random Zoom (0.1), Random Contrast (0.15), and Random Brightness (0.1).
- **Imbalance Handling:** Minority classes were manually oversampled, combined with a dynamically computed `class_weight` dictionary and a Sparse Categorical Focal Loss function ($\gamma = 2.0$) to heavily penalize confident misclassifications.

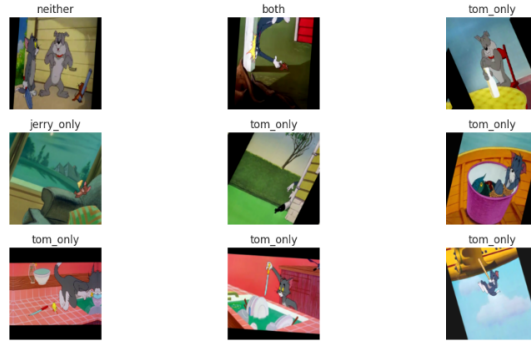


Figure 3: Augmented Training Samples

3. Architecture & Training Strategy

- **Model Architecture:** Transfer learning was leveraged using an `EfficientNetB0` backbone (ImageNet weights). The custom head utilized Global Average Pooling, a `Dense(128)` layer with ReLU, spatial Dropouts (0.35 and 0.2), and a 4-class Softmax output. Images were standardized to $224 \times 224 \times 3$.
- **Two-Stage Training:**
 1. **Stage A (Head Training):** Backbone frozen. Trained for 20 epochs using Adam ($lr = 1e - 3$).
 2. **Stage B (Fine-Tuning):** Top 30 backbone layers unfrozen. Fine-tuned for 12 epochs using a reduced learning rate ($lr = 1e - 5$) to adapt pre-trained features to the cartoon domain without catastrophic forgetting.
- **Callbacks:** Early Stopping (patience=5, restore best weights) and ReduceLROnPlateau were utilized to monitor validation loss and stabilize convergence.

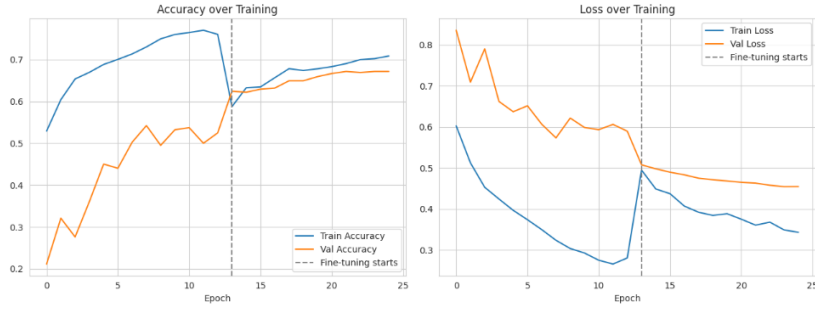


Figure 4: Accuracy and Loss Across Both Training Stages

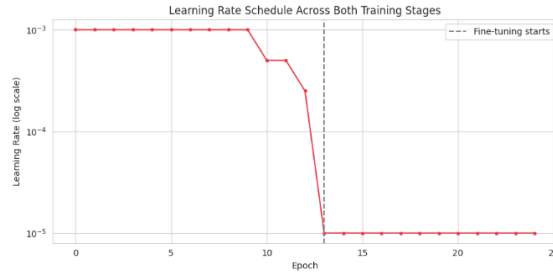


Figure 5: Learning Rate Schedule (Log Scale)

4. Evaluation & Interpretability

- **Validation Performance:** The model achieved a Validation Macro F1-score of 0.5710. The `tom_only` and `jerry_only` classes achieved F1-scores of 0.78 and 0.76 respectively. The network struggled most with the `both` class (F1-score of 0.34) due to overlapping character bounding boxes and extreme data scarcity.
- **Interpretability (Grad-CAM):** Gradient-weighted Class Activation Mapping (Grad-CAM) was utilized as a visual sanity check. Heatmaps confirmed the model successfully learned to isolate the characters rather than relying on background artifacts.

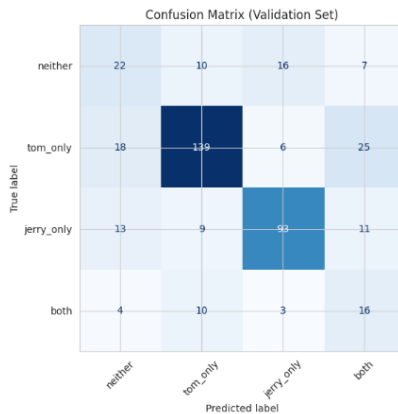


Figure 6: Confusion Matrix (Validation Set)

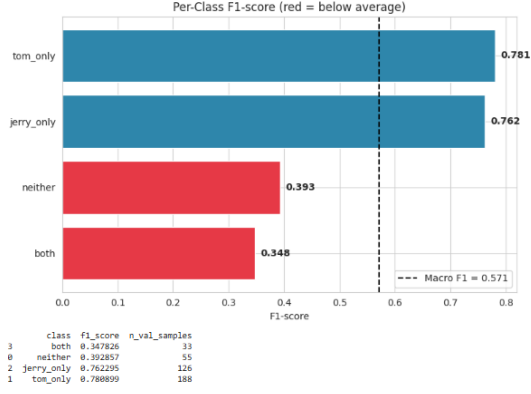


Figure 7: Per-Class F1-Score Breakdown

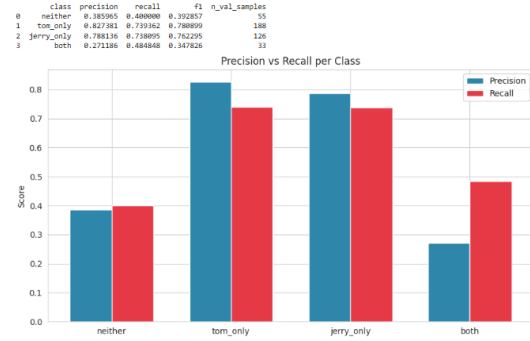


Figure 8: Grad-CAM Attention Maps (Correct Predictions)

5. Inference & Key Findings

- **Test-Time Augmentation (TTA):** Final test predictions were generated using TTA to ensure robustness against image noise. The model executed one clean pass and five augmented passes per test image, averaging the probabilities for the final submission.
- **Observations:** Stacked imbalance handling (oversampling + class weights + Focal Loss) may have introduced gradient instability for the hardest examples. For future iterations, implementing multi-subject augmentations like CutMix or MixUp is recommended to directly improve the network’s ability to classify the `both` class.
- **Reproducibility:** All operations were strictly deterministic using `RANDOM.STATE = 42`.



Figure 9: Sample Test Predictions via Test-Time Augmentation